// Program: Merge Sort vs Multithreaded Merge Sort

```
import java.util.Arrays;
import java.util.Scanner;

class MergeSort {
    // Standard Merge Sort
    public static void mergeSort(int[] arr, int left, int right) {
        if (left < right) {
            int mid = left + (right - left) / 2;

            mergeSort(arr, left, mid);
            mergeSort(arr, mid + 1, right);

            merge(arr, left, mid, right);
        }
    }
    static void merge(int[] arr, int left, int mid, int right) {
        int n1 = mid - left + 1;
        int n2 = right - mid;

        int[] L = new int[n1];
        int[] R = new int[n2];

        System.arraycopy(arr, left, L, 0, n1);
        System.arraycopy(arr, mid + 1, R, 0, n2);

        int i = 0, j = 0, k = left;
        while (i < n1 && j < n2) {
            if (L[i] <= R[j]) arr[k++] = L[i++];
            else arr[k++] = R[j++];
        }
        while (i < n1) arr[k++] = L[i++];
        while (j < n2) arr[k++] = R[j++];
    }
}

// Multithreaded Merge Sort
class ThreadedMergeSort extends Thread {
    private int[] arr;
    private int left, right;
    private static final int THRESHOLD = 1000; // use normal merge sort for small
arrays

    ThreadedMergeSort(int[] arr, int left, int right) {
        this.arr = arr;
        this.left = left;
        this.right = right;
    }
    @Override
    public void run() {
        if (right - left < THRESHOLD) {
            MergeSort.mergeSort(arr, left, right);
        } else if (left < right) {
            int mid = left + (right - left) / 2;

            ThreadedMergeSort leftThread = new ThreadedMergeSort(arr, left, mid);
            ThreadedMergeSort rightThread = new ThreadedMergeSort(arr, mid + 1, right);

            leftThread.start();
            rightThread.start();
            try {
                leftThread.join();
                rightThread.join();
            } catch (InterruptedException e) {
```

```
                e.printStackTrace();
            }
            MergeSort.merge(arr, left, mid, right);
        }
    }
}
public class MergeSortComparison {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter size of array: ");
        int n = sc.nextInt();

        int[] arr = new int[n];
        System.out.println("Enter array elements:");
        for (int i = 0; i < n; i++) arr[i] = sc.nextInt();

        int[] arrCopy1 = Arrays.copyOf(arr, n);
        int[] arrCopy2 = Arrays.copyOf(arr, n);

        // Measure standard merge sort
        long startTime = System.nanoTime();
        MergeSort.mergeSort(arrCopy1, 0, n - 1);
        long endTime = System.nanoTime();
        double timeMergeSort = (endTime - startTime) / 1e6;

        // Measure multithreaded merge sort
        ThreadedMergeSort threadedSort = new ThreadedMergeSort(arrCopy2, 0, n - 1);
        long startThread = System.nanoTime();
        threadedSort.start();
        try {
            threadedSort.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        long endThread = System.nanoTime();
        double timeThreaded = (endThread - startThread) / 1e6;

        System.out.println("\nSorted array (Merge Sort): " +
Arrays.toString(arrCopy1));
        System.out.println("Time taken (Merge Sort): " + timeMergeSort + " ms");

        System.out.println("\nSorted array (Multithreaded Merge Sort): " +
Arrays.toString(arrCopy2));
        System.out.println("Time taken (Multithreaded Merge Sort): " + timeThreaded + "
ms");

        sc.close();
    }
}
```

Output:

```
Enter size of array: 6
Enter array elements:
10 7 8 9 1 5

Sorted array (Merge Sort): [1, 5, 7, 8, 9, 10]
Time taken (Merge Sort): 0.947 ms

Sorted array (Multithreaded Merge Sort): [1, 5, 7, 8, 9, 10]
Time taken (Multithreaded Merge Sort): 0.4146 ms
```

Performance Analysis

1. Time Complexity

Algorithm	Best Case	Worst Case	Average Case
Merge Sort	O(n log n)	O(n log n)	O(n log n)
Multithreaded Merge Sort	O(n log n)	O(n log n)	O(n log n)

Note: Multithreading may **reduce actual wall-clock time** on multi-core systems but **does not change asymptotic complexity**.

2. Space Complexity

Algorithm	Space Complexity
Merge Sort	O(n) (auxiliary array)
Multithreaded Merge Sort	O(n) + O(log n) thread stack

3. Best Case vs Worst Case

- **Best Case:** Array is already sorted. Merge sort still divides and merges → O(n log n)
- **Worst Case:** Array is reverse sorted → O(n log n)
- **Observation:** Merge sort is **stable and predictable**, unlike quick sort.
- **Multithreading:** Faster in practice on large arrays due to parallelism, especially if array is large enough to utilize multiple cores.

4. Sample Input/Output

Input:

```
Enter size of array: 6
Enter array elements:
10 7 8 9 1 5
```

Output:

```
Sorted array (Merge Sort): [1, 5, 7, 8, 9, 10]
Time taken (Merge Sort): 0.947 ms

Sorted array (Multithreaded Merge Sort): [1, 5, 7, 8, 9, 10]
Time taken (Multithreaded Merge Sort): 0.4146 ms
```